QA Engineer I - Technical Assessment

Thank you for your interest in the **QA Engineer I** position. This repository contains a small terminal-based **Currency Converter** (Java 17 + Lanterna) and an acceptance-test harness built on **Cucumber + TestNG**. Your job is to design test cases for the converter and then automate them.

This is an **open-book** assessment. You may search the web, read documentation and use AI assistants. What we evaluate is your judgement: the test cases you choose, the quality of your automation, and how well you can **explain** what you built and why.

Table of contents

1. Getting started
2. Requirements under test
3. Phase 1 - Write the test cases
4. Phase 2 - PD Team review
5. Phase 3 - Automate with Cucumber
6. Phase 4 - Explain your solution
7. Submission checklist
8. How you will be evaluated
9. Rules and FAQ

---

1. Getting started

Prerequisites

Tool	Version
JDK	17+
Maven	3.8+
Git	any recent version

Clone, build and run

```
git clone <this-repo-url>
cd <repo>
git checkout -b candidate/<your-name>

mvn clean package           # compiles and runs the unit tests
./scripts/run.sh            # launches the GUI (scripts\run.bat on Windows)
mvn verify -P acceptance    # runs the Cucumber acceptance tests
```

The Cucumber HTML report is written to `target/cucumber-report.html`.

Recommended reading before you start

File	Why it matters
`docs/user-manual.md`	How the application behaves from a user's point of view
`docs/design.md`, `docs/architecture.md`	How the application is built
`src/main/java/.../model/Currency.java`	Supported currencies and their rates
`src/main/java/.../model/ConversionResult.java`	The result object your tests must verify

What is already provided in `src/test/java/com/komatsu/currencyconverter/acceptance/`

File	Status	Purpose
`AcceptanceRunner.java`	Provided - do not modify	Cucumber/TestNG entry point
`Hooks.java`	Provided	Fixes the locale and closes the GUI after every scenario
`ScenarioContext.java`	Provided	State shared between step classes (injected by PicoContainer)
`ConverterDriver.java`	Provided	Drives the real GUI on a headless virtual terminal (enter amount, select currencies, wait for result)
`ConversionAssessor.java`	Provided	Helper that validates inputs, counts decimal places and builds a `ConversionResult`
`InputSteps.java`	**You complete**	Step definitions for input validation, amount entry and dropdown selection
`ConversionSteps.java`	**You complete**	Step definitions that perform conversions and verify results
`src/test/resources/features/currency_conversion.feature`	**You complete**	Your Gherkin scenarios

You may add helper methods or fields if you need them, but **keep your step definitions in the two step classes** above and explain any change you make to the provided classes.

---

2. Requirements under test

Your test cases must cover **every** requirement below. Each test case should trace back to at least one requirement ID.

Input validation

ID	Requirement
R1	Throw `IllegalArgumentException` if `amount <= 0`
R2	Throw `IllegalArgumentException` if the source or target currency code is `null` or empty

Conversion flow

ID	Requirement
R3	Input the amount into the amount field
R4	Select the source currency from the dropdown
R5	Select the target currency from the dropdown
R6	Wait for the result container to become visible
R7	Extract the converted amount and the exchange rate
R8	Format the input amount to 2 decimal places using `String.format("%.2f", amount)`
R9	Count decimal places in both the formatted input amount string and the result string

Validations

ID	Requirement
R10	**Accuracy:** `Math.abs(conversionResult - amount * rate) <= 0.01`
R11	**Decimal precision:** the result has 2 or more decimal places
R12	**Dynamic update:** modify the input amount, confirm the result updates by comparing before/after values, then reset the GUI to the original amount to ensure a clean state

Returned `ConversionResult` - all fields populated

ID	Field	Expected value
R13	`inputAmount`	The exact amount entered into the amount field (e.g. `100.0`)
R14	`sourceCurrency`	The code selected in the source dropdown (e.g. `"USD"`)
R15	`targetCurrency`	The code selected in the target dropdown (e.g. `"EUR"`)
R16	`conversionResult`	The converted amount extracted from the result field in the UI (e.g. `92.35`)
R17	`inputDecimalPlaces`	Decimal places in the formatted input string (e.g. `2` for `"100.00"`)
R18	`resultDecimalPlaces`	Decimal places in the result string displayed in the UI (e.g. `2` for `"92.35"`)
R19	`conversionAccurate`	`true` only if `Math.abs(conversionResult - amount * rate) <= 0.01`
R20	`decimalPrecisionMaintained`	`true` only if the result has 2 or more decimal places
R21	`resultUpdatedOnChange`	`true` only if the result updates when the input changes
R22	`conversionComplete`	`true` only if **all** of the above conditions are met

3. Phase 1 - Write the test cases

Create the file `docs/test-cases.md` containing your manual test cases.

Required format

Use one entry per test case with at least these fields:

Field	Description
TC ID	Unique identifier, e.g. `TC-01`
Title	Short, descriptive name
Requirement(s)	IDs from section 2 that this test covers (e.g. `R1`, `R10`)
Type	Positive / Negative / Boundary / Edge
Priority	High / Medium / Low
Preconditions	State required before execution
Test data	Amount, source currency, target currency, etc.
Steps	Numbered, unambiguous steps
Expected result	Observable, verifiable outcome

Example (structure only - write your own content):

```
### TC-XX - <title>
- **Requirement(s):** R?, R?
- **Type:** Positive
- **Priority:** High
- **Preconditions:** ...
- **Test data:** amount = ..., source = ..., target = ...
- **Steps:**
  1. ...
  2. ...
- **Expected result:** ...
```

Also include in `docs/test-cases.md`

1. **Traceability matrix** - a table mapping every requirement **R1-R22** to the test cases that cover it. Every requirement must be covered at least once.
2. **Assumptions and questions** - anything ambiguous you noticed in the requirements and how you chose to interpret it.

Things to think about

- Positive, negative **and** boundary values (what is the smallest valid amount? what about `0`?

negative numbers? very large numbers? amounts with many decimals?).

- Different currency pairs, including the same currency as source and target.
- Currencies with very different rates (e.g. `JPY` vs `GBP`).
- How the GUI state should look *after* a test (clean-state requirement in R12).
- Avoid duplicating tests that verify exactly the same thing - quality over quantity.

> Commit Phase 1 on its own (e.g. `docs: add currency converter test cases`) so reviewers can see your test design before your automation.

4. Phase 2 - PD Team review

1. Push your branch and open a **Pull Request** titled `QA Engineer I - Test cases - <Your Name>` containing **only** `docs/test-cases.md`.
2. The **PD Team** will review the test cases and may leave comments or request changes.
3. Address every comment, either by updating the test cases or by replying with your reasoning.
4. **Do not start Phase 3 until the PD Team approves your test cases.**

5. Phase 3 - Automate with Cucumber

Once your test cases are approved, automate them.

5.1 Feature file - `src/test/resources/features/currency_conversion.feature`

- Write one or more **Scenarios** / **Scenario Outlines** per approved test case.
- Tag each scenario with its test case ID and type, e.g. `@TC-01 @positive @R10`.
- Use **Scenario Outline + Examples** for data-driven cases (multiple amounts / currency pairs).
- Use a **Background** for shared setup where it makes sense.
- Write steps in business language (what the user does and sees), not implementation details.

5.2 Step definitions

Class	Responsibility
`InputSteps.java`	`@Given` / `@When` steps that launch the GUI, validate inputs (R1, R2), enter the amount and select currencies (R3-R5), plus `@Then` steps for validation errors
`ConversionSteps.java`	`@When` steps that perform/assess the conversion and `@Then` steps that verify accuracy, precision, dynamic update and every `ConversionResult` field (R6-R22)

Guidelines:

- Share state between the two classes **only** through `ScenarioContext` (it is injected via the constructor by PicoContainer - no static fields).
- Create the `ConverterDriver` and `ConversionAssessor` lazily in a step and store them in the context; `Hooks` will close the GUI after each scenario.
- Use TestNG assertions (`assertEquals`, `assertTrue`, ...) with **meaningful failure messages**.
- Use a delta when comparing `double` values.
- Keep steps reusable and parameterised (Cucumber expressions such as `{double}`, `{string}`).
- No `Thread.sleep` in step definitions - use the waiting provided by `ConverterDriver`.

5.3 Definition of done

- [ ] `mvn verify -P acceptance` passes locally with **zero** failing, pending or undefined steps.
- [ ] Every approved test case is automated (or you explain why it cannot be).
- [ ] Scenarios are tagged with their TC IDs so traceability is preserved.
- [ ] No unused imports, commented-out code or debug `System.out.println`.
- [ ] Code is formatted consistently with the rest of the project.

---

6. Phase 4 - Explain your solution

Create `SOLUTION.md` at the root of the repository covering:

1. **Approach** - how you analysed the requirements and designed your test cases.
2. **Coverage** - which requirements are covered by which scenarios, and any gaps.
3. **Automation design** - how your feature file and step definitions are organised, and why.
4. **How to run** - exact commands to execute your tests and where to find the report.
5. **Defects / observations** - any bugs, odd behaviour or requirement ambiguities you found.
6. **Use of AI and external resources** - which tools/sites you used and for what. Using AI is allowed; **not disclosing it is not**.
7. **What you would improve** with more time.

You will also be asked to **walk through your solution live** with the team (about 20-30 minutes). Be ready to explain any line of code you submitted and to make a small change on the spot.

---

7. Submission checklist

Your final Pull Request (`QA Engineer I - Automation - <Your Name>`) should contain:

- [ ] `docs/test-cases.md` - approved test cases + traceability matrix
- [ ] `src/test/resources/features/currency_conversion.feature` - completed
- [ ] `src/test/java/com/komatsu/currencyconverter/acceptance/InputSteps.java` - completed
- [ ] `src/test/java/com/komatsu/currencyconverter/acceptance/ConversionSteps.java` - completed
- [ ] `SOLUTION.md` - explanation of your solution
- [ ] Clean commit history with descriptive messages
- [ ] Green build: `mvn clean verify -P acceptance`

---

8. How you will be evaluated

Area	Weight	What we look for
Test design	30%	Full requirement coverage, positive/negative/boundary thinking, clear and reproducible steps, traceability
Automation quality	30%	Readable Gherkin, reusable and well-organised steps, correct assertions, clean state, green build
Communication	25%	Quality of `SOLUTION.md`, PR descriptions, responses to review feedback, live walkthrough
Engineering practices	15%	Git hygiene, code style, no dead code, sensible commits

---

9. Rules and FAQ

Can I use Google, Stack Overflow, documentation or AI tools? Yes. This is an open-book test. Just disclose what you used in `SOLUTION.md` and make sure you understand everything you submit - you will be asked about it.

Can I modify the application code under `src/main`? No. You are testing the application, not changing it. If you find a bug, document it in `SOLUTION.md` (and, if appropriate, write a scenario that exposes it).

Can I modify the provided test helpers (`ConverterDriver`, `ConversionAssessor`, `ScenarioContext`, `Hooks`)? Prefer not to. If you think a change is necessary, keep it minimal and justify it in `SOLUTION.md`.

Can I add new dependencies to `pom.xml`? No. Everything you need is already included.

A requirement seems ambiguous - what should I do? Make a reasonable assumption, write it down in the *Assumptions and questions* section of `docs/test-cases.md`, and move on. You may also raise it as a question in your Phase 1 PR.

How long should this take? Plan for roughly **4-6 hours** of work in total. Contact the recruiter if you need more time.

Good luck - we look forward to reviewing your work!